

Q1/

man → to see manual

\$ man ls

\$ man cd

ls → list file and directory

\$ ls

\$ ls1

cd → change working directory

\$ cd

\$ cd :/ directory name

pwd → print working directory

\$ pwd

mkdir → make directory

\$ mkdir directory name

rmdir → remove directory

\$ rmdir :/ directory name

cat → display file content

\$ cat file.txt

less → pager program used to view the content

\$ less

head → display new lines at a file

\$ head -n 1

\$ head -n 10

tail → display few lines from last at a file

\$ tail -n 2

2023/3/02

CSE - B1

Date
Page

CP → Copy the file - Src-Dst

\$ cp

\$ cp

\$ cp

mv → move file

\$ mv txt directory name

rm → remove file or directory

\$ rm txt

touch → create an empty file

\$ touch text.txt

echo → out the string that is passed

\$ echo "Hello World"

\$ echo text.txt

Ping → \$ ping google.com

chmod → changes access mode of a file or directory

\$ chmod

\$ chmod -w hello.txt

chown → change ownership

\$ chown root hello.txt

Exit → terminates the current program

\$ exit

2023/3/02

CSE - B1

Date
Page

Grep → search pattern in a file

\$ grep "hello"

\$ grep -i "hello"

\$ grep -e [XZ] 00.txt

diff → show difference between two files

\$ diff hello.txt hello2.txt

PS → show working process

\$ ps

\$ ps -f

\$ ps -p 4000

top → table of process

kill → kill process

Sudo → run as admin

Shutdown → shutdown the system

WC → word count

\$ wc hello.txt

Q2) (i) CPU info

Cat / Proc / cpuinfo

Cat / Proc / cpuinfo / grep "model name"

Cat / Proc / cpuinfo / grep "CPU core"

Cat / Proc / cpuinfo / grep "CPU Cores"

(ii) memory info

Vim Stel

free -m

Cat / Proc / meminfo

Q3) (i) cd

include <stdio.h>

include <sys/stat.h>

include <sys/types.h>

include <syslib.h>

int main (int argc, char *argv[])

if (argc > 2)

printf("usage: %s <dir name>\n",

argv[0])

return 1;

int status = dir (argv[1]);

if (status == -1) printf("%s directory changed\n",

else printf("%s directory not changed\n",

char * cwd = getcwd (NULL, 0);

if (cwd == NULL)

printf("get cwd failed\n");

return 1;

}

else

printf("%s directory not changed\n",

return 0;

ii) ls

include <stdio.h>

include <string.h>

include <sys/types.h>

include <dirent.h>

int main () { int argc, char *argv[]

DIR *dir;

struct dirent *entry;

bool list = false;

char *font = "usage: %s <dir> <path name>";

if (argc > 2) { if (argc[1] == "-l")

int len = strlen (argv[1]);

if (len > 2)

if (argv[1][2] == 'h') show all true;

else if (argv[1][2] == 'l') list = true;

else printf ("font: %s\n",

return 0;

if (len > 2) { if (argv[1][2] == 'h') show all

else if (argv[1][2] == 'l') list = true;

2023/3/02

CSE B1

classmate

Date
Page

else {

Print f (fpath, args[2]);

return 2;

}

} // for network <1> //

if (argc < 2 || argc > 3) {

printf ("Usage: %s\n", argv[0]);

return 1;

return 3;

}

while (fgets (line, sizeof line, fp) != NULL) {

if (strlen (line) < 1) {

printf ("Continue\n");

printf ("Usage: %s\n", argv[0]);

if (strlen (line) < 1) {

printf ("Usage: %s\n", argv[0]);

fclose (fp);

return 0;

}

} //

#include <stdio.h>

#include <sys/stat.h>

#include <sys/types.h>

int main (int argc, char **argv) {

if (argc < 2) {

printf ("Usage: %s\n", argv[0]);

return 1;

2023/3/02

CSE B1

classmate

Date
Page

int status = mkdir (args[1], S_IRWXU);

if (status != 0) {

printf ("Error: %s\n", args[1]);

}

} //

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

void main (const char *filename, const char *keyword) {

FILE *file = fopen (filename, "r");

if (file == NULL) {

printf ("Error: opening file\n");

exit (EXIT_FAILURE);

char line[1024];

int line_number = 0;

while (fgets (line, sizeof line, file) != NULL) {

line_number++;

if (strstr (line, keyword) != NULL) {

printf ("Line %d: %s\n", line_number, line);

fclose (file);

int main (int argc, char **argv) {

if (argc < 2) {

printf ("Usage: %s\n", argv[0]);

return 1;

char *filename = argv[1];

return 0;

}

Q4) The above C program demonstrates the use of Command-line Arguments. This program calculates the Sum of integers passed as Command line arguments.

```
#include <stdio.h>
#include <stdlib.h>
int main (int argc, char *argv[]) {
```

```
if (argc < 2) {
    printf("usage: %s <numbers> <numbers>...\n", argv[0]);
    return 1;
}
```

```
int sum = 0;
```

```
for (int i = 1; i < argc; i++)
```

```
sum += atoi(argv[i]);
```

```
}
```

```
printf("The sum of the numbers is %d\n", sum);
```

```
return 0;
```

Q5)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <limits.h>
```

```
void Swap(int *a, int *b) {
```

```
int temp = *a;
```

```
*a = *b;
```

```
*b = temp;
```

```
}
```

```
void SelectionSort(int arr[], int n) {
```

```
start = clock(); for (int i = 0; i < n-1; i++) {
```

```
int mini = i;
```

```
for (int j = i+1; j < n; j++)
```

```
if (arr[j] < arr[mini]) mini = j;
```

```
}
```

```
if (mini != i) Swap(&arr[i], &arr[mini]);
```

```
}
```

```
end = clock();
```

```
double time = end - start;
```

```
printf("Time of execution of Selection Sort: %.2f\n", time);
```

}

```
void BubbleSort(int arr, int n) {
```

```
start = clock();
```

```
for (int i = 0; i < n-1; i++)
```

2023/02

Date

Page

CSE 132

```

for (int i = 0; i < n-1; i++) {
    if (arr[i] > arr[i+1]) {
        swap(arr[i], arr[i+1]);
    }
}

```

```

}
end = clock();

```

```

double time = end - start;

```

```

printf("Execution time for Insertion Sort: %.1f", time);

```

}

```

void Insertion Sort (int arr[], int n) {

```

```

    int start = clock();

```

```

    for (int i = 1; i < n; i++) {

```

```

        int j = i-1;

```

```

        while (j > 0 && arr[j] > arr[j+1]) {

```

```

            arr[j+1] = arr[j];

```

```

        }
    }

```

```

    arr[j+1] = arr[i];

```

```

    arr[j+1] = arr[i];

```

}

```

end = clock();

```

```

double time = end - start;

```

```

printf("Execution Time for Insertion Sort: %.1f", time);

```

}